

A RAG layer over the Obsidian vault

How it works, and what it actually buys

Chirayu Gupta — 2026-08-13

Built on **obsidian-second-brain** (MIT) · embeddings via local Ollama `bge-m3`

The problem

When you say *"why did the limit come out wrong?"*, I have to find the note that answers it. Without an index, my only tool is `grep`.

That fails in a specific way: **you rarely remember the word the note uses.**

What you ask

- "the yield was fine but the uncertainty blew up"
- "totals came from the wrong place"
- "how much signal survives the charm cut"

What the note says

- `autoMCStats`, `amc@NLO` negative weights
- `sumw`, `parquet metadata`, `sidecar json`
- `≥1 c-jet`, `medium PNet WP`, `23.1%`

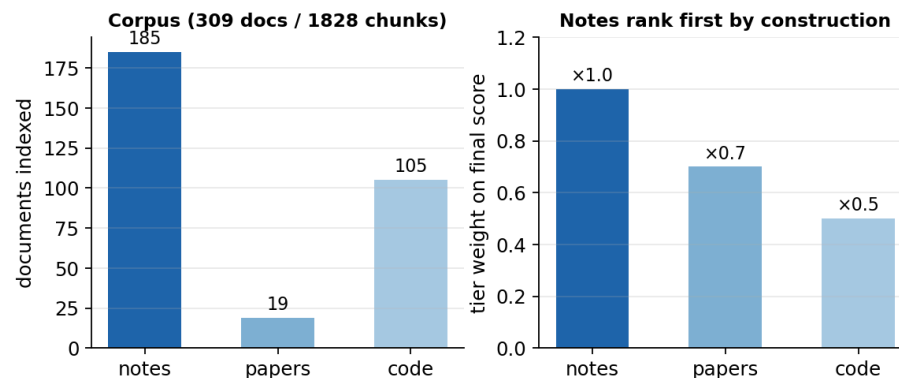
Zero lexical overlap. `grep` cannot bridge that gap — it matches characters, not meaning.

What was built

```
scripts/rag/vault-search \  
  "why did the limit come out wrong"  
  
scripts/rag/vault-search --tier code \  
  "how are condor jobs resubmitted"
```

One command. Three tiers, searched together and ranked so **notes come first**.

Documented in `CLAUDE.md`, so every future session knows to reach for it before grepping.



How it works — two rankings, fused

1. Semantic

Every document → a 1024-d vector (`bge-m3` , local Ollama).

Your query → same space. Rank by cosine similarity.

Long docs are **chunked**; a document scores by its *best* chunk, so one relevant section surfaces a 40-page paper.

2. Keyword

`grep` over the query's content words, **normalised by document length** and boosted when the term appears in the filename.

Fusion — Reciprocal Rank Fusion

$$\text{score} = \sum_{r \in \{sem, kw\}} \frac{1}{60 + \text{rank}_r}$$

RRF combines the two **rank orders**, never the raw scores — a cosine of 0.57 and a match-count of 19 are not comparable quantities.

Then shaped by tier weight × recency.

Why both? They fail in *opposite* directions.

Semantic finds paraphrases but drifts on short factual queries. Keyword nails exact terms but is blind to meaning.

Ranking: notes first, by construction

Tier	Source	Weight
notes	vault markdown	1.0
papers	References/**/*.pdf	0.7
code	lxplus repos via sshfs	0.5

Tier weight **multiplies** the fused score, so an equally-relevant note outranks a code file — one blended list, not three separate ones.

Recency is a gentle tiebreak: 900-day half-life, floor 0.70. A fresh note wins among equals; a 2-year-old note stays findable.

Papers are searched by content, not filename:

"reweighting negative weight events to fix MC statistics"

→ 2510.16217-negweight-reweighting.pdf **(rank 1)**

Text extracted with `pdftotext`, chunked, embedded.

Code is read over the sshfs mount — Ollama runs on the laptop, so lxplus code is indexed *from* the laptop.

"how are condor jobs submitted and resubmitted"

→ `jobs_status.py`, `submit_condor.py` **(ranks 1-2)**

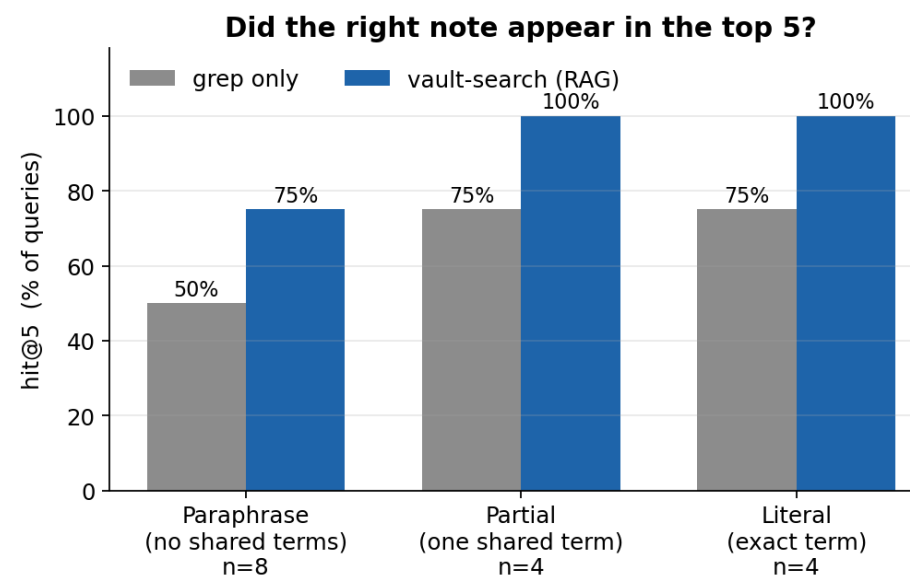
Evaluation: 16 queries, labelled ground truth

Queries are phrased the way you'd actually ask — deliberately **not** using the note's own vocabulary.

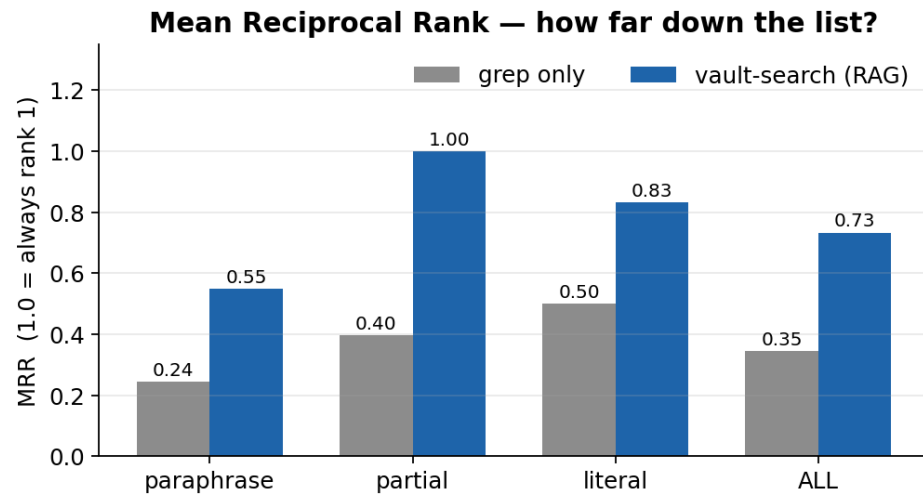
Class	n	Definition
paraphrase	8	no distinctive term shared
partial	4	one common term shared
literal	4	names the exact term

hit@5 — correct note in top 5

MRR — 1.0 means always rank 1



Retrieval quality



	grep	RAG
hit@5 (all)	62%	88%
MRR (all)	0.35	0.73
MRR paraphrase	0.24	0.55
MRR partial	0.40	1.00

MRR 0.73 vs 0.35 — with the RAG the right note is usually *rank 1*; with grep it's typically 3rd or absent.

On `partial` and `literal` queries the correct note is now **always** the top hit.

But hit@5 is the wrong benchmark

Without the RAG I don't run one grep. I run a *search loop*:

guess a term → grep → read a file → wrong → guess again → read another.

Every wrong file I open is context burned. That's the real cost.

So the honest comparison is what it costs **me** to answer your question:

no RAG

grep content words → if nothing, retry with the rarest term → read candidates in rank order until the right note appears (give up after 4)

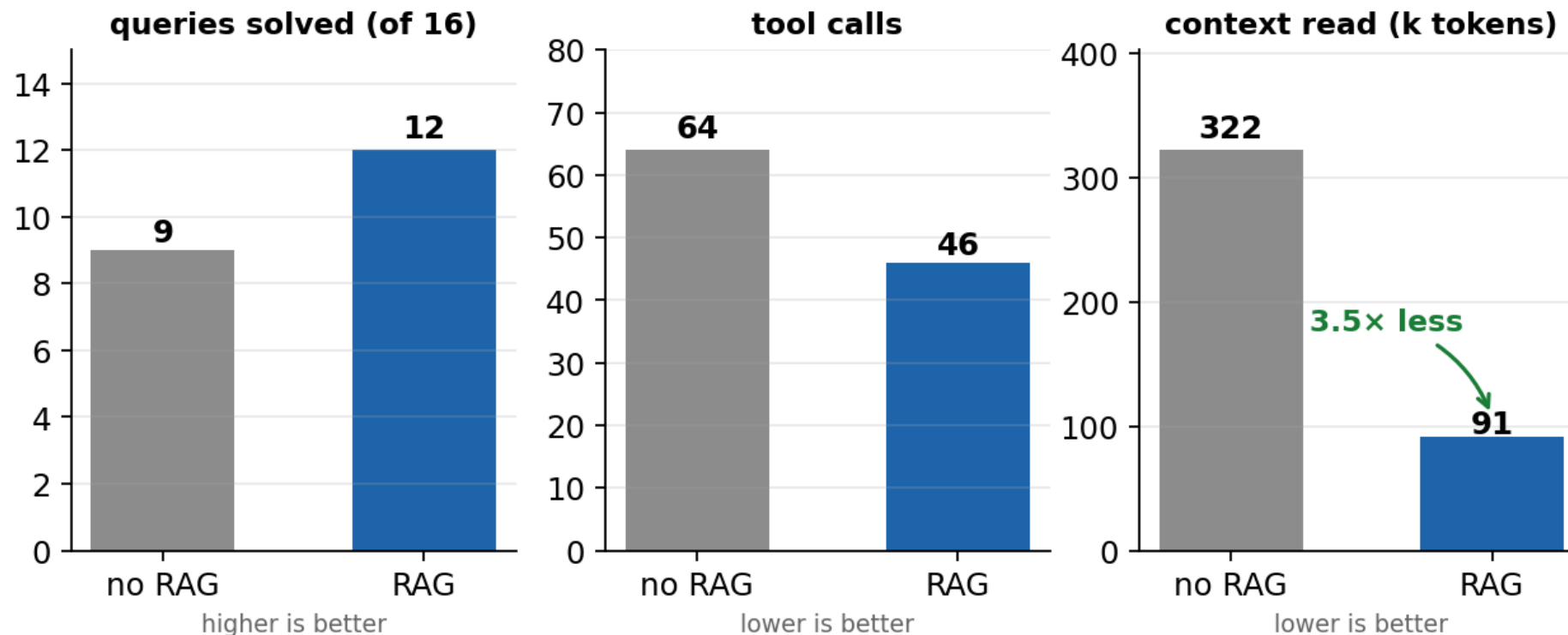
RAG

one `vault-search` call → read candidates in rank order (give up after 4)

Cost = tool calls + bytes of every file read along the way (~4 bytes/token). Both counted; neither estimated away.

The result that matters

Cost to the agent of answering from the vault

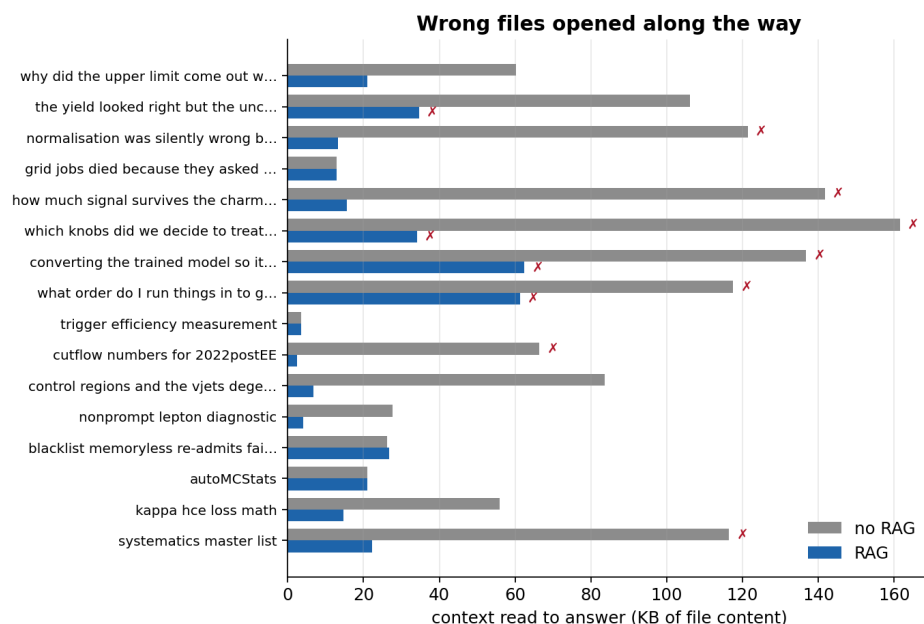


3.5× less context burned

322k → 91k tokens across 16 queries

More queries solved (9 → 12), in **fewer** tool calls (64 → 46). The win isn't accuracy — it's **not reading the wrong files.**

Per-query, including the losses



The RAG is **not** uniformly better:

- **Query 2** "*yield looked right but uncertainty inflated*" — grep found it, RAG missed. The note's framing is numeric, not linguistic.
- **Query 13** "*blacklist...*" — grep ranked it 2nd, RAG 3rd. Rare exact terms are grep's home turf.

4 of 16 remain unsolved by both. Those are notes whose content genuinely doesn't resemble the question asked.

rg is still the right tool when you know the exact string. This layer is for when you don't.

The third baseline: no vault at all

So far both arms assume **the notes exist**. What about a cold agent — no vault, no notes, no index — asked *"why did the limit come out wrong?"* It must **re-derive the finding** from the repo: 168 source files, 7.5 MB, 856 commits.

Modelled, not measured

I cannot un-know your notes, so this arm is a model.

Every input is stated so it can be argued with:

Input	Value	Source
repo source	7.5 MB / 168 files	measured
orientation	15% of source read	assumed
re-derive one finding	180k tok, 60 calls	anchored ↓
not derivable at all	35%	assumed

Why 180k tokens is *conservative*

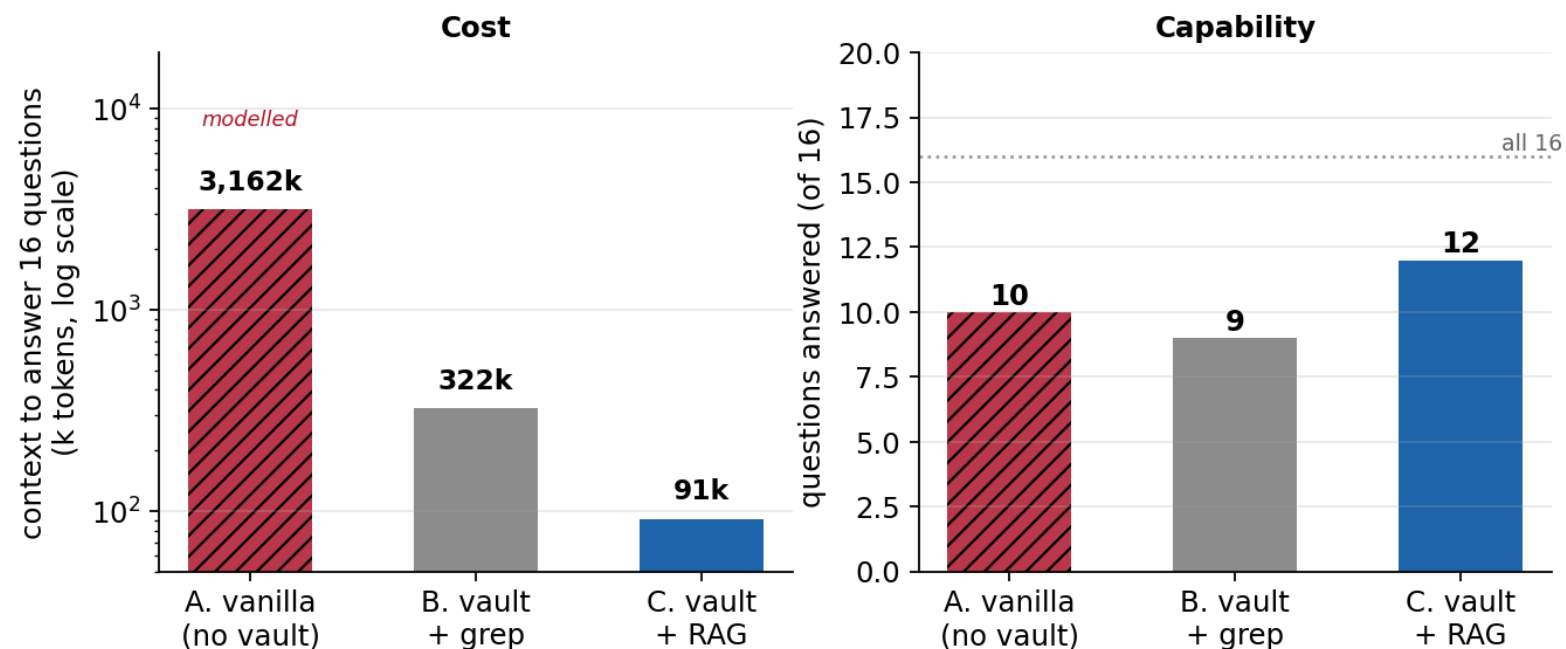
The autoMCStats thread in your vault runs **2026-06-17** → **2026-07-12** — about four weeks, five notes. `sumw-normalization-trap.md` alone is 277 lines of hard-won conclusions.

180k tokens ≈ **one long session**. The real investigation took weeks of human work.

The vanilla number is understated, not inflated.

Three-way comparison

Vanilla agent vs vault vs vault+RAG

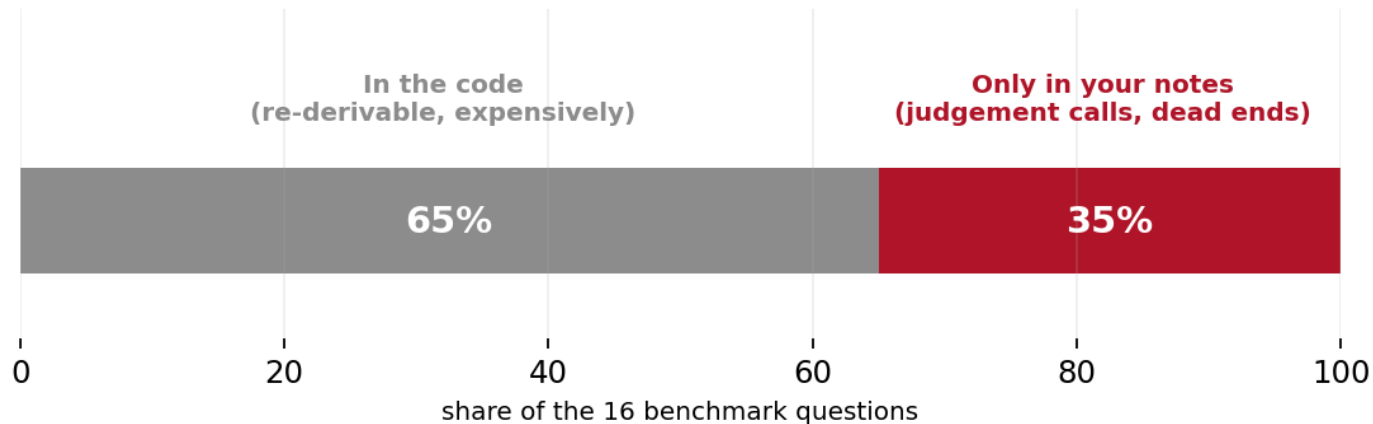


	tokens	vs RAG
A. vanilla (<i>modelled</i>)	~3,162k	35×
B. vault + grep	322k	3.5×
C. vault + RAG	91k	—

Note the **right panel**: vanilla answers *fewer* questions than grep-on-the-vault, while burning 35× the context. **More tokens does not buy the answer.**

What the token ratio hides

A third of the questions have no answer in the source code



No answer exists in the source code, at any budget:

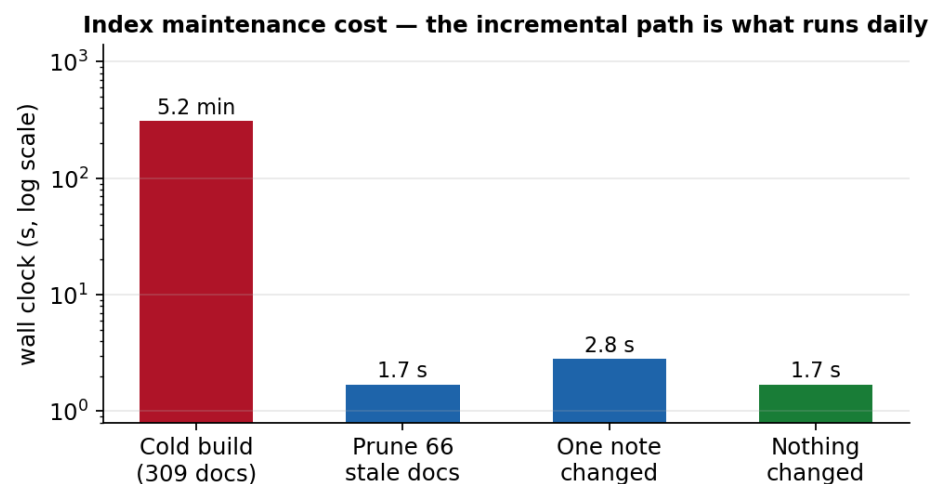
- *"which knobs did we treat as one nuisance?"* → a **judgement call**. Code shows what was done, never what was considered and rejected.
- *"yield looked right but uncertainty inflated"* → a **symptom you observed**, not a code path.

This is the real argument for the vault — not retrieval speed.

Dead ends and "we tried X, it failed because Y" leave **no artifact** in the repo. A cold agent re-derives the *code*; it cannot re-derive the *decisions*.

The RAG makes that record cheap to reach — it does not create it. You do, by writing it down.

Keeping the index honest



Incremental by mtime+size — matters because the code tier is read over sshfs.

Prunes actively: entries whose source file is gone are dropped (verified: added a file, indexed it, deleted it → 1 pruned).

Hooks: SessionEnd + PreCompact fire a rebuild.

The hook **detaches and returns in 8 ms** — SessionEnd has a 1.5 s shared budget a real rebuild would blow.

A flock stops concurrent sessions double-embedding.

What is *not* indexed

The code tier initially swallowed **65 JSON data files** — generated `partitions.json` (XRootD path lists), correctionlib lookup tables, fileset dumps. Machine-generated, useless for semantic recall, and they churn — so they'd force constant re-embedding.

Excluded

- `condor/`, `out/`, `img/` job trees
- `partitions*.json`, `analysis/data/*.json`
- `filesets/`, `.sites_map.json`
- `higgscharm_thomas/` (1.2 GB collaborator checkout)

Result: code tier 171 → 105 docs, while indexed `.py` files rose 60 → 68 — the old

`condor/` walk had been masking real scripts.

Structurally impossible to index

`CODE_EXTS` is an **allowlist** — only

`.py .cc .h .C .cpp .sh .yaml .yml .json .cfg`
`.md`

are ever read.

`.root`, `.parquet`, `.pkl`, `.onnx`, `.npy` can never enter the index, wherever they sit.

Two bugs worth naming

Both were found by testing, and both were **silent** — the tool returned plausible results while a whole component was dead. That is the dangerous failure mode.

1. The keyword arm was never running

`shutil.which("rg")` returned `None` — `rg` in this shell is a *zsh function*, not a binary. Every result was semantic-only for the first half of the build.

Fix: fall back to `/usr/bin/grep`; make the failure loud on stderr.

2. Raw match counts buried exact titles

"*systematics master list*" did **not** find
`2026-07-24-systematics-master-list.md`.

A long note saying "list" 19× outranked it; semantic had it at rank 1, RRF averaged it to 24.

Fix: normalise by $\sqrt{\text{size}}$, boost filename matches. hit@5 62% → 88%.

Honest limitations

This is a retrieval benchmark, not an end-to-end agent study. I did not run paired Claude Code sessions on real tasks — that needs controlled variables I can't hold fixed here. The token numbers model a search loop; they are not measured session transcripts.

n = 16 queries, one vault, ground truth labelled by me. Directionally sound, not a publication-grade result.

On lxplus there is no semantic layer at all. Ollama runs laptop-side and the index is gitignored, so `vault-search` there degrades to keyword-only — verified, with a clear stderr note, on Python 3.9.

The retrieval quality is downstream of note quality. Your notes open with the conclusion ("*the parquets ARE self-normalizing, but...*"). That is why semantic search works here at all. The index is not a substitute for writing things down well.

Summary

How it works

Semantic (`bge-m3`) + keyword, fused by RRF,
shaped by tier weight × recency.
309 docs / 1828 chunks / 38 MB, gitignored.
Self-pruning, hook-refreshed, 8 ms to fire.

What it buys

hit@5 **62%** → **88%**, MRR **0.35** → **0.73**.
Context read to answer: **322k** → **91k tokens**.
Paraphrased questions go from coin-flip to
reliable.

The bigger effect is the vault, not the index. A cold agent burns ~35× the context and still answers *fewer* questions — because a third of them are judgement calls that were never in the code. The RAG makes that record cheap to reach; writing it down is what makes it exist.

`scripts/rag/` · built on **obsidian-second-brain** (MIT, pinned `4d5b673`) — search core only, ~450 lines vendored, zero third-party
Python deps